

Tarot Appointment System

API Documentation

Overview

Welcome to the API documentation for the **Tarot Appointment System**. This API allows clients to manage appointments, schedules, services, and user authentication for a seamless tarot reading experience.

Features

- User Authentication (Login, Registration)
- Schedule Management
- Appointment Booking
- Service Management
- Role-Based Access Control
- Appointment Status Updates

TABLE OF CONTENTS

	Page
Register	
POST Register	3-4
POST Login	4
GET Detail	5
GET Account	6
Appointment	
POST Add Appointment	7
GET Appointment	8
PUT Complete Status	9
GET Pending Appointment	10
GET Complete Appointment	11
Roles	
POST Roles	12
GET Roles	13
DELETE Roles	14
POST Assign Roles	15
Schedule	
GET Schedule	16
POST Add Schedule	17
PATCH Update Schedule	18
DELETE Schedule	19
Services	
GET Services	20
POST Services	21
PATCH Update Services	22
Delete Services	23
Schemas	24-25

Register

This API is responsible for managing user accounts, including registration, login, and retrieving user details. When a user signs up, they provide their email, full name, and password. They are automatically assigned a "User" role unless specified otherwise. Logging in requires the correct email and password, after which the system provides a secure access token (like a digital key) that lets users interact with protected features.

Account

POST `/api/Account/register` Registers a new user in the system.

Sample Request:
POST /api/account/register Authorization: None Content-Type: application/json

```
{  "Email": "user@example.com",  "FullName": "John Doe",  "Password": "strongpassword",  "Roles": ["User"]}
```

Sample Response:
Status Code: 200 OK Content-Type: application/json

```
{  "IsSuccess": true,  "Message": "Account Created Successfully!"}
```

Error Responses:

- 400 Bad Request: If the input data is incomplete or invalid.
- 500 Internal Server Error: If the registration process fails.

Notes:

- The registered user will be assigned the "User" role by default if no roles are provided.

Parameters

Try it out

No parameters

Request body

application/json

The registration details, including email, password, and roles.
Example Value | Schema

```
{  "email": "user@example.com",  "fullName": "string",  "password": "string",  "roles": [    "string"  ]}
```

Responses

Code	Description	Links
200	Success	No links

Media type

text/plain

Controls Accept header.

Example Value | Schema

```
string
```

POST

/api/Account/login

Authenticates a user and generates a JWT token.

Sample Request:

POST /api/account/login Authorization: None Content-Type: application/json

```
{  "Email": "user@example.com",  "Password": "strongpassword"}
```

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{  "Token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  "IsSuccess": true,  "Message": "Login Success.",  "Roles": ["User"]}
```

Error Responses:

- 401 Unauthorized: If the user is not found or the password is incorrect.
- 400 Bad Request: If the input data is invalid.

Notes:

- This endpoint generates a JWT token for the authenticated user.
- The user's roles are returned along with the token.

Parameters

Try it out

No parameters

Request body

application/json

The login details, including email and password.

Example Value | Schema

```
{  "email": "string",  "password": "string"}
```

Responses

Code	Description	Links
200	Success	No links

Media type

text/plain

Controls Accept header.

Example Value | Schema

```
{  "token": "string",  "roles": [    "string"  ],  "IsSuccess": true,  "message": "string"}
```

GET

/api/Account/detail

Retrieves the details of the authenticated user.

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{  "Id": "12345",  "Email": "user@example.com",  "FullName": "John Doe",  "Roles": ["User"],  "PhoneNumber": "+1234567890",  "PhoneNumberConfirmed": true}
```

Error Responses:

- 404 Not Found: If the authenticated user is not found.

Notes:

- This endpoint retrieves the details of the currently authenticated user.

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Success Media type text/plain Controls Accept header. Example Value Schema <pre>{ "id": "string", "fullName": "string", "email": "string", "roles": ["string"], "phoneNumber": "string", "twoFactorEnabled": true, "phoneNumberConfirmed": true, "accessFailedCount": 0}</pre>	No links

GET

/api/Account

Retrieves a list of all users in the system along with their details and roles.

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
[
  {
    "Id": "12345",
    "Email": "user1@example.com",
    "FullName": "User One",
    "Roles": ["Admin"]
  },
  {
    "Id": "67890",
    "Email": "user2@example.com",
    "FullName": "User Two",
    "Roles": ["User"]
  }
]
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated.

Notes:

- This endpoint retrieves a list of all users in the system along with their roles.

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Success	No links

Media type

text/plain

Controls Accept header.

Example Value

Schema

```
[
  {
    "id": "string",
    "fullName": "string",
    "email": "string",
    "roles": [
      "string"
    ],
    "phoneNumber": "string",
    "twoFactorEnabled": true,
    "phoneNumberConfirmed": true,
    "accessFailedCount": 0
  }
]
```

Appointment

This API allows users to book, view, and manage their tarot reading appointments. It ensures secure access by requiring users to log in and restricting certain actions based on user roles.

Appointment

POST `/api/Appointment/AddAppointment` Adds a new appointment for the authenticated user. Only accessible by users with the "User" role.

Sample Request:
POST /api/Appointment/AddAppointment Authorization: Bearer {your_token_here} Content-Type: application/json

```
{  "schedule_id": 1,  "service_id": 2,  "date_appointment": "2025-01-15",  "time_slot": "10:00 AM"}
```

Sample Response:
Status Code: 200 OK Content-Type: application/json

```
{  "appointment_id": 101,  "service_id": 2,  "schedule_id": 1,  "date_appointment": "2025-01-15",  "time_slot": "10:00 AM"}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated.
- 400 Bad Request: If the user ID is invalid, no available slots, or the input data is incomplete.
- 404 Not Found: If the specified schedule does not exist.

Notes:

- This endpoint is accessible only to users with the "User" role.
- The authenticated user's ID is used to associate the appointment with the user.
- The number of available slots is decremented when an appointment is successfully created.

Parameters

Try it out

No parameters

Request body

application/json

The data transfer object containing the appointment details.

Example Value Schema

```
{  "appointment_id": 0,  "service_id": 0,  "schedule_id": 0,  "date_appointment": {    "year": 0,    "month": 0,    "day": 0,    "dayOfWeek": 0  },  "time_slot": "string",  "status": 0,  "user_fullname": "string"}
```

Responses

Code	Description	Links
200	Success	No links

GET

/api/Appointment/GetAppointment

Gets the list of appointments for the authenticated user.

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{
  "UserId": "12345",
  "Appointments": [
    {
      "appointment_id": 1,
      "service_id": 2,
      "date_appointment": "2025-01-15",
      "time_slot": "10:00 AM",
      "status": 0
    },
    {
      "appointment_id": 2,
      "service_id": 3,
      "date_appointment": "2025-01-16",
      "time_slot": "11:00 AM",
      "status": 1
    }
  ]
}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated.

Notes:

- This endpoint returns all appointments associated with the authenticated user.
- The returned appointments are filtered based on the authenticated user's ID.

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Success	No links

PUT

/api/Appointment/CompleteStatus/{appointmentId}

Updates the status of the specified appointment to "complete" (status = 1). Only accessible by users with the "Admin" role.

Sample Request:

PUT /api/Appointment/CompleteStatus/{appointmentId} Authorization: Bearer {your_token_here} Content-Type: application/json

{ }

Sample Response:

Status Code: 200 OK Content-Type: application/json

"Appointment status updated to complete."

Error Responses:

- 401 Unauthorized: If the user is not authenticated.
- 403 Forbidden: If the user does not have the "Admin" role.
- 404 Not Found: If the appointment ID does not exist.

Notes:

- This endpoint is restricted to users with the "Admin" role.
- The appointment status is updated to "complete" (status = 1) when successfully processed.

Parameters

Try it out

Name	Description
appointmentid integer(\$int32) (path) required	The unique identifier of the appointment to be updated.

appointmentid

Responses

Code	Description	Links
200	Success	No links

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{
  "UserId": "12345",
  "Appointments": [
    {
      "appointment_id": 1,
      "service_id": 2,
      "date_appointment": "2025-01-15",
      "time_slot": "10:00 AM",
      "user_fullname": "John Doe"
    }
  ]
}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated.
- 403 Forbidden: If the user does not have the "Admin" role.

Notes:

- This endpoint is restricted to users with the "Admin" role.
- Only pending appointments (status = 0) will be returned.

Parameters		Try it out
No parameters		
Responses		
Code	Description	Links
200	Success	No links

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{
  "UserId": "12345",
  "Appointments": [
    {
      "appointment_id": 1,
      "service_id": 2,
      "date_appointment": "2025-01-15",
      "time_slot": "10:00 AM",
      "user_fullname": "John Doe"
    }
  ]
}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated.
- 403 Forbidden: If the user does not have the "Admin" role.

Notes:

- This endpoint is restricted to users with the "Admin" role.
- Only completed appointments (status = 1) will be returned.

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Success	No links

Roles

This API helps an admin manage user roles in an application. It allows the admin to create, view, delete, and assign roles to users. Only authorized admins can perform these actions to maintain security. The system prevents errors like duplicate roles or assigning a role to a non-existent user. This ensures proper user access and organization within the application.

Roles

POST

/api/Roles

Creates a new role in the application. Only accessible by users with the "Admin" role.

Sample Data:

• Request Body:

```
{  "RoleName": "Manager"}
```

• Response (Success):

```
{  "message": "Role Created successfully"}
```

• Response (Failure):

```
{  "message": "Role creation failed."}
```

Parameters

Try it out

No parameters

Request body

application/json

The data transfer object containing the role name.

Example ValueSchema

```
{  "roleName": "string"}
```

Responses

Code	Description	Links
200	Success	No links

GET

/api/Roles

Retrieves a list of all roles in the application, along with the total number of users assigned to each role. Only accessible by users with the "Admin" role.

Sample Data:

• Response:

```
[
  {
    "Id": "1",
    "Name": "Admin",
    "TotalUsers": 3
  },
  {
    "Id": "2",
    "Name": "User",
    "TotalUsers": 15
  },
  {
    "Id": "3",
    "Name": "Manager",
    "TotalUsers": 5
  }
]
```

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Success	No links

Media type

text/plain

Controls Accept header.

Example Value | Schema

```
[
  {
    "id": "string",
    "name": "string",
    "totalUsers": 0
  }
]
```

DELETE

/api/Roles/{id}

Deletes an existing role by its ID. Only accessible by users with the "Admin" role.



Sample Data:

- Request: `DELETE /api/Roles/{id}`
- Response (Success):

```
{  "message": "Role deleted successfully."}
```
- Response (Failure):

```
{  "message": "Role deletion failed."}
```

Parameters

Try it out

Name	Description
Id required string (path)	The ID of the role to be deleted.
id	

Responses

Code	Description	Links
200	Success	No links

POST

/api/Roles/assign

Assigns a role to a user in the application. Only accessible by users with the "Admin" role.

Sample Data:

• Request Body:

```
{  "UserId": "userId123",  "RoleId": "roleId456"}  
```

• Response (Success):

```
{  "message": "Role assigned successfully"}  
```

• Response (Failure):

```
{  "message": "Role assignment failed."}  
```

Parameters

Try it out

No parameters

Request body

application/json

Request body

application/json

The data transfer object containing the user ID and the role ID to be assigned.

Example Value

Schema

```
{  "userId": "string",  "roleId": "string"}  
```

Responses

Code	Description	Links
200	Success	No links

Schedule

This API lets users view their schedules, while only admins can create, update, or delete them. Each request requires authentication, meaning users must log in to get a unique token. The system retrieves schedules from a database and responds with structured data, like available slots and dates. If an unauthorized user tries to access or modify schedules, the system blocks their request.

Schedule

GET

/api/Schedule/GetSchedule

Retrieves the list of schedules for the authenticated user. Accessible by users with the "User" or "Admin" role.

Sample Request:

GET /api/Schedule/GetSchedule Authorization: Bearer {your_token_here}

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{  "userId": "12345678-abcd-efgh-ijkl-901234567890",  "schedules": [    {      "schedule_id": 1,      "number_slots": 10,      "date": "2025-01-14"    },    {      "schedule_id": 2,      "number_slots": 5,      "date": "2025-01-15"    }  ]}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated or authorized to perform the action.

Notes:

- This endpoint is accessible to users with the "User" or "Admin" roles.
- The `userId` field in the response represents the ID of the authenticated user.

Parameters

No parameters

Try it out

Responses

Code	Description	Links
200	Success	No links

POST

/api/Schedule/AddSchedule

Adds a new schedule to the application. Only accessible by users with the "Admin" role.

Sample Request:

POST /api/Schedule/AddSchedule Authorization: Bearer {your_token_here} Content-Type: application/json

```
{
  "number_slots": 10,
  "date": "2025-01-14"
}
```

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{
  "schedule_id": 1,
  "number_slots": 10,
  "date": "2025-01-14"
}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated.
- 400 Bad Request: If the user ID is invalid or the input data is incomplete.

Notes:

- This endpoint is accessible only to users with the "Admin" role.
- The authenticated user's ID is used to associate the schedule with the user.

Parameters

Try it out

No parameters

Request body

application/json

The data transfer object containing the schedule details.

Example Value Schema

```
{
  "schedule_id": 0,
  "number_slots": 0,
  "date": {
    "year": 0,
    "month": 0,
    "day": 0,
    "dayOfWeek": 0
  }
}
```

Responses

Code	Description	Links
200	Success	No links

PATCH

/api/Schedule/UpdateSchedule/{schedule_id}

Updates an existing schedule. Only accessible by users with the "Admin" role.

Sample Request:

PATCH /api/Schedule/UpdateSchedule/{schedule_id} Authorization: Bearer (your_token_here) Content-Type: application/json

```
{
  "number_slots": 5,
  "date": "2025-01-15"
}
```

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{
  "userId": "12345678-abcd-efgh-ijkl-901234567890",
  "updatedSchedule": {
    "schedule_id": 1,
    "number_slots": 5,
    "date": "2025-01-15"
  }
}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated.
- 404 Not Found: If the specified schedule does not exist.

Notes:

- This endpoint is accessible only to users with the "Admin" role.
- The `userId` field represents the ID of the authenticated user making the request.

Parameters

Try it out

Name	Description
schedule_id * required integer(\$int32) (path)	The ID of the schedule to be updated.

schedule_id

Request body

application/json

The data transfer object containing the updated schedule details.

Example Value | Schema

```
{
  "schedule_id": 0,
  "number_slots": 0,
  "date": {
    "year": 0,
    "month": 0,
    "day": 0,
    "dayOfWeek": 0
  }
}
```

Responses

Code	Description	Links
200	Success	No links

DELETE

/api/Schedule/DeleteSchedule/{schedule_id} Deletes an existing schedule. Only accessible by users with the "Admin" role.

Sample Request:

DELETE /api/Schedule/DeleteSchedule/{schedule_id} Authorization: Bearer {your_token_here}

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{  "userId": "12345678-abcd-efgh-ijkl-901234567890",  "scheduleId": 1,  "message": "Schedule deleted successfully."}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated.
- 404 Not Found: If the specified schedule does not exist.

Notes:

- This endpoint is accessible only to users with the "Admin" role.
- The `userId` field represents the ID of the authenticated user making the request.

Parameters

Try it out

Name	Description
schedule_id required integer(\$int32) (path)	The ID of the schedule to be deleted.

schedule_id

Responses

Code	Description	Links
200	Success	No links

Services

This API ensures that only authorized users can access or modify services, with admins having full control while regular users can only view available services. It includes endpoints to list, add, update, and delete services, requiring authentication via a token. Each action interacts with a database, storing and retrieving service details such as name, description, and price. This system helps maintain secure and organized service management for the application while restricting unauthorized access.

Service

GET

/api/Service/GetService

Retrieves the list of services for the authenticated user. Accessible by users with "Admin" or "User" roles.

Sample Request:

```
GET /api/Service/GetService Authorization: Bearer (your_token_here)
```

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{
  "userId": "12345678-abcd-efgh-ijkl-901234567890",
  "services": [
    {
      "service_id": 1,
      "service_name": "Tarot Reading",
      "description": "A detailed tarot card reading session.",
      "price": 50.0
    },
    {
      "service_id": 2,
      "service_name": "Astrology Consultation",
      "description": "An in-depth astrology chart analysis.",
      "price": 70.0
    }
  ]
}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated or authorized to perform the action.

Notes:

- This endpoint is accessible to users with the roles "Admin" or "User".
- The `userId` field in the response is the ID of the authenticated user making the request.

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Success	No links

POST

/api/Service/AddService

Adds a new service to the application. Only accessible by users with the "Admin" role.

Sample Request:

POST /api/Service/AddService Content-Type: application/json Authorization: Bearer (your_token_here)

```
{
  "service_name": "Tarot Reading Basic",
  "description": "A basic tarot card reading session.",
  "price": 50.0
}
```

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{
  "service_id": 1,
  "service_name": "Tarot Reading Basic",
  "description": "A basic tarot card reading session.",
  "price": 50.0
}
```

Error Responses:

- 401 Unauthorized: If the user is not authenticated.
- 400 Bad Request: If the provided user ID is invalid.

Parameters

Try it out

No parameters

Request body

application/json

The data transfer object containing the service details.

Example Value | Schema

```
{
  "service_id": 0,
  "service_name": "string",
  "description": "string",
  "price": "string"
}
```

Responses

Code	Description	Links
200	Success	No links

PATCH

/api/Service/UpdateService/{service_id}

Updates an existing service. Only accessible by users with the "Admin" role.

Sample Request:

PATCH /api/Service/UpdateService/{service_id} Content-Type: application/json Authorization: Bearer {your_token_here}

```
{
  "service_name": "Updated Tarot Reading",
  "description": "An updated description for the tarot reading service.",
  "price": 75.0
}
```

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{
  "userId": "12345678-abcd-efgh-ijkl-901234567890",
  "updatedService": {
    "service_id": 1,
    "service_name": "Updated Tarot Reading",
    "description": "An updated description for the tarot reading service.",
    "price": 75.0
  }
}
```

Error Responses:

- 404 Not Found: If the service with the given ID does not exist.
- 401 Unauthorized: If the user is not authenticated or not authorized to perform the action.

Parameters

Try it out

Name	Description
service_id * required	The ID of the service to be updated.
integer(\$int32) (path)	service_id

Request body

application/json

The data transfer object containing the updated service details.

Example Value

Schema

```
{
  "service_id": 0,
  "service_name": "string",
  "description": "string",
  "price": "string"
}
```

Responses

Code	Description	Links
200	Success	No links

DELETE

/api/Service/DeleteService/{service_id} Deletes an existing service. Only accessible by users with the "Admin" role.

Sample Request:

DELETE /api/Service/DeleteService/{service_id} Authorization: Bearer {your_token_here}

Sample Response:

Status Code: 200 OK Content-Type: application/json

```
{
  "userId": "12345678-abcd-efgh-ijkl-901234567890",
  "serviceId": 1,
  "message": "Service deleted successfully."
}
```

Error Responses:

- 404 Not Found: If the service with the given ID does not exist.
- 401 Unauthorized: If the user is not authenticated or not authorized to perform the action.

Parameters

Try it out

Name	Description
service_id * required	The ID of the service to be deleted.
integer(\$int32) (path)	
	service_id

Responses

Code	Description	Links
200	Success	No links

```
AppointmentDto ▾ {
  appointment_id integer($int32)
  service_id integer($int32)
  schedule_id integer($int32)
  nullable: true
  date_appointment DateOnly ▸ {...}
  time_slot string
  nullable: true
  status integer($int32)
  user_fullname string
  nullable: true
}
```

```
AuthResponseDto ▾ {
  token string
  nullable: true
  roles ▸ [...]
  isSuccess boolean
  message string
  nullable: true
}
```

```
CreateRoleDto ▾ {
  roleName* string
  minLength: 1
}
```

```
DateOnly ▾ {
  year integer($int32)
  month integer($int32)
  day integer($int32)
  dayOfWeek DayOfWeek integer($int32)
  Enum:
    ▸ Array [ 7 ]
  dayOfYear integer($int32)
  readOnly: true
  dayNumber integer($int32)
  readOnly: true
}
```

```
DayOfWeek ▾ integer($int32)
Enum:
  ▾ [ 0, 1, 2, 3, 4, 5, 6 ]
```

```
LoginDto ▾ {
  email* string
  minLength: 1
  password* string
  minLength: 1
}
```

```
RegisterDto ▾ {
  email* string($email)
  minLength: 1
  fullName* string
  minLength: 1
  password string
  nullable: true
  roles ▸ [...]
}
```

```
RoleAssignDto ▾ {
  userId string
  nullable: true
  roleId string
  nullable: true
}
```

```
RoleResponseDto ▾ {
  id string
  nullable: true
  name string
  nullable: true
  totalUsers integer($int32)
}
```



```
ScheduleDto ▾ {  
  schedule_id      integer($int32)  
  number_slots     integer($int32)  
  date             DateOnly > {...}  
}
```

```
ServiceDto ▾ {  
  service_id      integer($int32)  
  service_name    string  
                 nullable: true  
  description     string  
                 nullable: true  
  price           string  
                 nullable: true  
}
```

```
UserDetailDto ▾ {  
  id              string  
                 nullable: true  
  fullName        string  
                 nullable: true  
  email           string  
                 nullable: true  
  roles           > [...]  
  phoneNumber     string  
                 nullable: true  
  twoFactorEnabled boolean  
  phoneNumberConfirmed boolean  
  accessFailedCount integer($int32)  
}
```